

Game Theory Simulation

Project Document

CS-A1123 – Programming 2

Adi Dasgupta

Student Number: 103552167

Study Program: Exchange Studies

May 2026

Contents

1	General Information	4
2	General Description	4
3	User Instructions	4
3.1	Launching the Program	4
3.2	Playing the Game	5
3.3	Running Unit Tests	5
4	External Libraries	5
5	Structure of the Program	6
5.1	Class Overview	6
5.2	Class Diagram	7
5.3	Central Methods	7
6	Algorithms	7
6.1	Market Price Calculation	7
6.2	Profit Calculation	8
6.3	AI Best-Response Strategy	8
6.4	Nash Equilibrium Computation	8
6.5	Stochastic Game Termination	9
7	Data Structures	9
8	Files	9
8.1	Results File	10
8.2	No Input Files	10
9	Testing	10
9.1	Unit Tests	10
9.2	Manual Testing	11
9.3	Comparison to Testing Plan	11
10	Known Shortcomings and Flaws	11
11	3 Best and 3 Weakest Parts	12
11.1	Best Parts	12
11.2	Weakest Parts	12

12 Changes to the Original Plan	13
13 Realized Order and Schedule	13
14 Assessment of the Final Result	13
15 References	14

1 General Information

Title	Game Theory Simulation
Author	Adi Dasgupta
Student Number	103552167
Study Program	Exchange Studies
Date	May 2026

2 General Description

The program simulates a Cournot duopoly — a market structure where two firms produce the same product and simultaneously decide how much to produce each round. The market price is determined by a linear inverse demand curve that depends on the combined output of both firms. The player competes against an AI opponent and tries to maximise their cumulative profit over several rounds.

The game ends randomly after each round with a fixed probability, which adds a strategic uncertainty element that forces the player to think about both short and long-term decisions. At the end of the game, the program displays the Nash equilibrium quantities and prices so the player can compare their choices against theoretically optimal play.

The project has been implemented at the **hard difficulty level**. This includes all baseline requirements plus: stochastic game ending, a graphical user interface built with PyQt5, an AI opponent using best-response logic, Nash equilibrium comparison shown at the end, a live price history graph, and the ability to save results to a text file.

Compared to the project plan, the main difference is that the `ResultCalculator` class mentioned in the original class diagram was not implemented as a separate class. Instead, its functionality (profit calculation and equilibrium computation) was split between the `Firm` class and the `Market` class respectively, which felt more natural and kept the design cleaner.

3 User Instructions

3.1 Launching the Program

First make sure PyQt5 is installed:

```
pip install PyQt5
```

Then run the program from the project root directory:

```
python src/main_window.py
```

3.2 Playing the Game

When the window opens, the player sees:

- A **quantity input field** on the left (spinner, range 0–100)
- A **Submit button** to confirm your production quantity for that round
- **Info labels** showing the current round number, market price, AI quantity, your round profit, and cumulative total profit
- A **Nash equilibrium hint** showing the theoretically optimal quantity and price
- A **price history graph** on the right that updates after each round
- A **round history table** listing all past rounds with quantities, prices, and profits

Each round the player picks a quantity and presses Submit. The AI responds using a best-response strategy based on the player's previous move. The market price is calculated and both profits are updated. After each round, there is a 20% chance the game ends randomly.

When the game ends, a popup appears showing:

- Total profit for both the player and the AI
- Average quantities chosen
- Nash equilibrium quantity and price for comparison

The **Save Results** button becomes available after the game ends and lets the player save a summary to a `.txt` file. The **New Game** button resets everything.

3.3 Running Unit Tests

From the project root:

```
python -m unittest discover -s test -t . -v
```

This runs all 18 unit tests across the four test files.

4 External Libraries

The only external library used is **PyQt5**, which is permitted by the course guidelines for graphical user interface development. All other imports (`random`, `os`, `sys`, `unittest`) are

from Python's standard library.

Matplotlib was explicitly listed as a prohibited library. The price graph was therefore implemented from scratch using PyQt5's `QPainter` class, which draws directly onto a widget canvas. This was more work than using a plotting library but it demonstrates a better understanding of how graphics work at a lower level.

5 Structure of the Program

The program is divided into six classes, each with a clearly defined responsibility. The separation was intentional to keep individual components testable and easy to modify independently.

5.1 Class Overview

Market handles the economic model. It stores the demand curve parameters (`a` and `b`) and provides two methods: one to calculate the market price given two production quantities, and one to compute the Nash equilibrium analytically. Keeping this separate from the other classes means the economic formulas are all in one place, which makes it easy to change the demand curve parameters without touching anything else.

Firm represents one participant in the game, either the player or the AI. It stores the firm's current quantity, marginal cost, cumulative profit, and round history. It has methods to set the quantity for a round and to calculate the profit given a market price.

AIStrategy encapsulates the AI opponent's decision logic. It takes the game history and returns a production quantity. Keeping this separate from **GameController** means the AI strategy can be changed or upgraded without touching the game flow logic.

GameController is the central coordinator. It holds instances of **Market**, two **Firm** objects (player and AI), and **AIStrategy**. It manages round progression, triggers the random game-ending check, and provides a summary of results at the end. It also handles saving results to a file.

PriceGraph is a custom PyQt5 widget that draws the price history as a line graph using `QPainter`. It receives a list of prices and redraws itself whenever the list is updated.

MainWindow is the top-level GUI class. It builds the interface layout, handles user input events (button clicks, spinner changes), calls `GameController.play_round()` each turn, updates all the display elements, and shows the end-of-game popup.

5.2 Class Diagram

The overall structure follows the one presented in the project plan. The only deviation is that `ResultCalculator` was not implemented as a standalone class — its responsibilities were distributed to `Market` (equilibrium) and `Firm` (profit calculation), which avoided an unnecessary class that would have just delegated to others.

5.3 Central Methods

- `Market.get_price(q1, q2)` — calculates market price from total quantity
- `Market.get_nash_equilibrium()` — analytically computes Cournot Nash equilibrium
- `Firm.calculate_profit(price)` — computes and accumulates round profit
- `AIStrategy.decide(history)` — returns AI production quantity for the round
- `GameController.play_round(player_quantity)` — runs one full round and returns the result
- `GameController.get_summary()` — returns final game statistics
- `GameController.save_results(filepath)` — writes results to a text file
- `PriceGraph.update_prices(prices)` — refreshes the graph with new data
- `MainWindow.play_round()` — handles the Submit button click and updates all UI elements

6 Algorithms

6.1 Market Price Calculation

The market price is determined by the linear inverse demand function:

$$P = a - b \cdot (q_1 + q_2)$$

where $a = 100$ is the demand intercept, $b = 1$ is the slope, and q_1, q_2 are the quantities produced by the player and AI respectively. The price is floored at zero to prevent negative prices in extreme cases.

This is the standard Cournot duopoly demand model. An alternative would have been a non-linear demand curve (e.g. $P = a/Q$), but the linear model is analytically tractable

and has a clean Nash equilibrium solution, which makes the end-of-game comparison more meaningful.

6.2 Profit Calculation

Each firm's profit in a round is:

$$\pi_i = (P - c) \cdot q_i$$

where c is the marginal cost (set to 0 in this implementation) and P is the market price. Cumulative profit is tracked across rounds.

6.3 AI Best-Response Strategy

The AI uses a best-response function derived from standard Cournot theory. Taking the derivative of firm 2's profit with respect to q_2 and setting it to zero gives:

$$q_2^* = \frac{a - c - b \cdot q_1}{2b}$$

In the first round, with no history available, the AI plays the Nash equilibrium quantity $\frac{a-c}{3b} \approx 33$. From round 2 onwards it responds to the player's most recent quantity. This makes the AI reactive rather than random, which creates a more interesting and educational opponent.

An alternative would have been a random AI or a fixed-quantity AI, but neither of those would reflect real economic strategy and they would make the Nash equilibrium comparison at the end less meaningful.

6.4 Nash Equilibrium Computation

In a symmetric Cournot duopoly with linear demand and equal marginal costs, the Nash equilibrium quantity per firm is:

$$q^* = \frac{a - c}{3b}$$

and the equilibrium price is:

$$P^* = a - b \cdot 2q^* = \frac{a + 2c}{3}$$

With $a = 100$, $b = 1$, $c = 0$, this gives $q^* \approx 33.33$ and $P^* \approx 33.33$. These values are computed analytically at the end of each game and displayed alongside the player's actual average quantity, letting them see how close they were to optimal play.

6.5 Stochastic Game Termination

After every round, the game ends with probability $p = 0.2$. This is implemented by drawing a uniform random number and comparing it to the threshold:

```
if random.random() < self.end_probability:
    self.game_over = True
```

This means the expected number of rounds is $1/p = 5$, but the actual number varies. This uncertainty forces the player to weigh immediate profit against longer-term strategic positioning, which is the core tension in a repeated game with uncertain horizon.

7 Data Structures

The main data structures used are Python lists and dictionaries.

Lists are used to store the round history (`GameController.history`) and each firm's per-round records (`Firm.history`). Lists are appropriate here because the number of rounds is not known in advance and can grow dynamically. Accessing the last element (for the AI's best-response calculation) is $O(1)$ with a list, which is all that's needed.

Dictionaries are used for each round result and for the final summary, because they let values be accessed by descriptive keys like "player_quantity" or "price" rather than by index. This makes the code more readable and less error-prone than using tuples or positional lists.

Simple numeric variables (`int`, `float`) are used for quantities, prices, and profits within a round.

No custom data structures were needed. The problem domain is relatively simple and Python's built-in types are sufficient. An alternative would have been to use named tuples or dataclasses for round results, which would give type hints and attribute access. That might be a good improvement for a larger project.

8 Files

The program handles two types of files.

8.1 Results File

When the player clicks "Save Results" after a game ends, the program writes a plain text file (.txt) with the following structure:

```
Game Theory Simulation - Results
=====

Total rounds: 5

Round    Your Qty    AI Qty    Price    Your Profit
-----
1         20         33       47.0     940.0
2         25         27       48.0     1200.0
...

=====
Your total profit: 3400.0
AI total profit: 2800.0
Your avg quantity: 22.5
Nash eq. quantity: 33.33
Nash eq. price: 33.33
```

The file location is chosen by the player through a standard file dialog. Text format was chosen because it is human-readable and easy to inspect without any special software.

8.2 No Input Files

The program does not read any input files. All parameters (demand intercept, slope, end probability, marginal cost) are hardcoded as default values in the class constructors. The plan mentioned the possibility of reading parameters from a config file, but this was not implemented since the hardcoded values work well for the simulation and adding file I/O for parameters would have added complexity without much benefit.

9 Testing

9.1 Unit Tests

18 unit tests were written across four test files:

- `test_market.py` (4 tests) — price at zero production, price decreasing with quantity, price never negative, Nash equilibrium values

- `test_firm.py` (5 tests) — initial profit is zero, profit calculation, profit accumulation across rounds, negative quantity raises error, reset clears state
- `test_ai_strategy.py` (3 tests) — first move is Nash equilibrium, best response to high player quantity, quantity never goes negative
- `test_game_controller.py` (6 tests) — round counter increments, price appears in result, player profit appears in result, game does not end with zero probability, reset clears history, save results creates a file

All 18 tests pass. They can be run with:

```
python -m unittest discover -s test -t . -v
```

9.2 Manual Testing

The GUI was tested manually by playing multiple full games. Things checked:

- Submitting quantity 0 and very high quantities (e.g. 100) — price floors at zero correctly
- Playing exactly at the Nash equilibrium quantity (33) for several rounds — profits and AI response behave as expected
- Verifying the end screen values match what the history table shows
- Saving results to a file and checking the content is correct
- Pressing New Game and verifying everything resets properly

9.3 Comparison to Testing Plan

The testing plan in the project plan mentioned testing price decrease with quantity, profit accuracy, and game termination probability. All of these were covered. The plan also mentioned testing invalid inputs — this is partially covered by the negative quantity test in `test_firm.py`. The GUI input field is a spinner widget with a range of 0 to 100, so invalid input is prevented at the UI level rather than caught after the fact.

10 Known Shortcomings and Flaws

- **AI only looks one round back.** The best-response strategy uses only the player's most recent quantity. A smarter AI might average the last few rounds or use a more sophisticated repeated-game strategy. This was a conscious simplification to keep the implementation manageable.

- **No config file for parameters.** The demand curve parameters and end probability are hardcoded. The project plan mentioned optionally reading these from a file, but this was not implemented. Adding this would make the simulation more flexible.
- **Price graph is basic.** The graph drawn by `PriceGraph` shows price history as a simple line but has no axis labels with values, no gridlines, and no legend. This makes it harder to read precisely. A more polished graph would include these.
- **The GUI layout could be improved.** On smaller screens the window can feel cramped. The layout was not tested extensively at different resolutions.
- **No input validation on the GUI beyond the spinner range.** The spinner prevents values outside 0–100, but there is no warning if the player submits 0 every round, which is a valid but uninteresting choice.

11 3 Best and 3 Weakest Parts

11.1 Best Parts

1. **Nash equilibrium hint during the game.** Showing the optimal quantity and price as a hint in the sidebar (not just at the end) lets the player understand the theory while playing, rather than only seeing it as a reveal at the end. This was a small addition but makes the simulation more educational.

2. **The `AIStrategy` class is fully decoupled.** Because the AI logic lives in its own class and receives the history as a parameter, it is easy to swap in a different strategy without changing anything else. A random AI, a cooperative AI, or a tit-for-tat AI could all be plugged in by just replacing this one class.

3. **The price graph using `QPainter`.** Drawing the graph without matplotlib required implementing the coordinate scaling manually, which was more work but resulted in a lightweight solution that has no external dependencies beyond PyQt5. It also demonstrated how drawing on a canvas works at a lower level.

11.2 Weakest Parts

1. **The price graph has no axis value labels.** The graph shows the shape of price changes clearly but does not display exact values on the axes. This makes it harder to read precisely. Fixing this would require adding text rendering for tick marks in `PriceGraph.paintEvent()`.

2. **No config file.** Hardcoded parameters make the simulation inflexible. If the demand

parameters are changed it requires modifying the source code. A simple `config.txt` file parsed at startup would fix this.

3. The AI only reacts to the previous round. A smarter opponent would make the educational value of the simulation higher. Implementing a strategy that tracks the player's average or detects patterns would have made the game more challenging.

12 Changes to the Original Plan

The main structural change was dropping the `ResultCalculator` class. In the plan it was shown as a separate class responsible for profit calculation and equilibrium computation. During implementation it became clear that these responsibilities fit more naturally into `Firm` (profit) and `Market` (equilibrium), so a dedicated calculator class would have just been a thin wrapper.

The save-to-file feature was added during implementation — it was mentioned as optional in the plan but ended up being implemented since it matched the file formats section of the plan.

The schedule in the plan described implementing core logic first, then the controller, then the GUI. That is roughly the order things were built in, so no major deviation there.

Time-wise, the project took approximately 20 hours in total, which is a bit more than originally estimated but not significantly over.

13 Realized Order and Schedule

Week	What was done
Week 1 (Feb 2026)	Project plan written and submitted
Week 2 (Mar 2026)	<code>Market</code> and <code>Firm</code> classes, unit tests for <code>Market</code>
Week 3 (Mar 2026)	<code>GameController</code> class, unit tests for <code>GameController</code>
Week 4 (Apr 2026)	<code>AIStrategy</code> , <code>MainWindow</code> , <code>PriceGraph</code> , save feature
Week 5 (May 2026)	Final testing, bug fixes, project document

14 Assessment of the Final Result

Overall the project meets all the hard difficulty requirements. The simulation works as intended — the game plays correctly, the AI responds sensibly, the Nash equilibrium comparison is shown at the end, and the GUI is functional and reasonably clean.

The weakest part of the result is the price graph, which is functional but visually basic. Given more time, improving the graph with proper axis labels and gridlines would have made the program more polished.

The class structure is clean and modular. Adding new features — a different AI strategy, a different demand model, or a multi-round tournament mode — would not require major restructuring. The separation between game logic and GUI means both can be tested and modified independently.

The biggest thing I would do differently is to implement the `ResultCalculator` class as originally planned, or alternatively use Python dataclasses for round results instead of plain dictionaries. This would have made the code slightly more structured.

15 References

1. Aalto University external study material on demand, supply, duopoly, and Nash equilibrium: http://aalto-econ.fi/tervio/kandimikro/external_materials.html
2. PyQt5 documentation — used for QMainWindow, QWidget, QPainter, QSpinBox, QTableWidget, QFileDialog: <https://www.riverbankcomputing.com/static/Docs/PyQt5/>
3. Python standard library documentation — random, os, sys, unittest: <https://docs.python.org/3/library/>
4. Varian, H.R. (1992). *Microeconomic Analysis*, 3rd edition. W. W. Norton. (Cournot duopoly and Nash equilibrium formulas)